nature machine intelligence

Perspective

https://doi.org/10.1038/s42256-026-01303-y

Characterizing the role of reasoning tokens in large language models as state over tokens

Received: 22 December 2025

Accepted: 20 August 2026

Published online: 7 October 2026

Check for updates

Mosh Levy [1] ✉, Zohar Elyoseph[2], Shauli Ravfogel[3] & Yoav Goldberg[1,4]

Large language models increasingly rely on generating reasoning tokens that appear to describe a reasoning process performed by the model to arrive at the answer. However, empirical evidence reveals that when read as text, these tokens are not a faithful explanation of the model's actual reasoning process. To address this gap between appearance and function, we introduce the State over Tokens (SoT) conceptual framework. SoT reframes reasoning tokens not as a linguistic narrative, but as an externalized computational state—the mechanism that persists the computation across the separate generation cycles. The SoT framing explains how the tokens can drive correct reasoning without being a faithful explanation when read as text and surfaces previously overlooked research questions on these tokens. We discuss how it dispels two common misconceptions: that the text records the complete process, and that the model uses the words with the meanings a human reader ascribes to them. We argue that this framing offers a fruitful way to think about these tokens, one that captures their mechanics accurately while remaining simple and intuitive, and that adopting it opens research directions. To better understand the mechanism of large-language-model reasoning, research should move beyond reading the tokens as text and also focus on decoding them as state.

The assertion that large language models (LLMs) can reason now appears unremarkable[1,2]. A key factor in achieving this was training the models to generate a sequence of tokens before their final answer, which substantially improves performance[3,4].

We refer to this sequence of symbols, which includes phrases such as 'therefore', 'consider' and 'it follows that', as the 'reasoning tokens', and explicitly distinguish this name from 'reasoning text', which is the same tokens when interpreted by a reader according to their English semantics. Treating the tokens as text is useful; if seen as derivations of the answer, they may help the reader verify individual steps and transitions between them, thus establishing trust in the answer[5–7].

The combination of (1) utility in improving the answer, (2) appearance as an English text, and (3) utility of the text as a derivation of the answer often leads to interpreting the derivation in the text as reflective of the process done by the model when deriving the answer. This is strengthened by metaphors such as 'chain-of-thought', which imply that the steps in the text are 'thoughts' that explain the process. However, there is a difference between a good derivation of the answer and an account of the process by which the model arrived at it—what we would call a faithful explanation. The reasoning text can provide the former, but falls short as the latter. Even the simple case of using the text to infer whether a hint planted in the input affected the output breaks down: the model can rely on the hint while omitting it from its reasoning text[8,9] (see more in the 'Plausibility without faithfulness' section).

While those findings clarify what the reasoning tokens are not, they leave a conceptual vacuum as to what they are. Our aim in this Perspective is to help fill that vacuum. Drawing on the idea that metaphors structure understanding and guide thinking[10], we believe that adopting more apt descriptions and metaphors can steer researchers and practitioners towards more fruitful directions and surface a new set of questions that are less salient under the prevailing view of the reasoning text as an explanation.

[1]Department of Computer Science, Bar-Ilan University, Ramat Gan, Israel. [2]School of Counseling and Human Development, The Faculty of Education, University of Haifa, Haifa, Israel. [3]NYU Center for Data Science, New York University, New York, NY, USA. [4]Allen Institute for AI, Seattle, WA, USA. ✉e-mail: moshe0110@gmail.com

https://doi.org/10.1038/s42256-026-01303-y

BOX 1

The whiteboard analogy

Consider a hypothetical scenario: you are placed in a room with a problem written on a whiteboard. Your task is to solve it, but under a peculiar constraint: every 10 seconds, your memory is completely wiped and resets to the same state as it was when you first entered the room. Within each interval, you can read what is on the board and add a single word. These rounds repeat until you finally write down the solution.

How might you solve a problem under such constraints? You may write intermediate results on the board—numbers, conclusions or partial computations—that you can use when you return after being 'reset'. You might perform several mental calculations before writing down just the result so the whiteboard may not capture every calculation that you did within each cycle. Moreover, you may use an encoding scheme when writing on the board: abbreviations, symbols or even apparent gibberish that will mean something specific to you when you encounter it in the next cycle. All in all, an outside observer may interpret the whiteboard text incorrectly.

To understand reasoning tokens, the appearance of which can be deceiving (as empirical research has found), we propose a framing that captures their functional role accurately while remaining simple and intuitive. To this end, we advocate viewing them as representing a computational state embedded as tokens, what we consider State over Tokens (SoT). This view characterizes the reasoning tokens as a computational device that enables the persistence of a process across separate computation cycles. We portray the reasoning tokens as state carriers of a computational process, a frame that captures the mechanism accurately while remaining simple enough to be used productively in research and in everyday discourse.

The whiteboard analogy in Box 1 mirrors the model's operation: the words are the reasoning tokens, you are the model and the 10-second interval represents the model's limited capacity per cycle. Motivated by this intuition, we present the SoT framework (see 'The State over Tokens framework' section) and use it to demonstrate two common misconceptions that underlie the belief that the text is a faithful explanation (see 'Two misconceptions behind the illusion of explanation' section) and to examine its distinctive ontological divergence (see 'The ontological divergence between text and state' section).

The SoT framework naturally gives rise to new research questions (see 'Questions raised by state over tokens' section), challenging current interpretability methods to account for multiple cycles, and questioning whether language is a special computational medium that can theoretically act both as a state and as a faithful explanation.

Finally, this perspective compels us to reconsider established metaphors. While 'chain-of-thought' captures the sequential nature of generation, it misleadingly suggests that the text contains the complete reasoning process; SoT clarifies that tokens are the state passed between computations, and do not necessarily express the computations themselves. Similarly, the 'scratchpad' metaphor obscures a crucial distinction: while humans use a scratchpad to supplement internal memory, for LLMs, the tokens are what allows any persistence between cycles.

By establishing reasoning tokens as externalized computational state, the SoT framework characterizes how LLMs use what seems to us as natural language as a computational medium, and provides a foundation for future work on decoding how models use language tokens to reason.

The remainder of the Perspective is structured as follows. The 'Reasoning tokens and reasoning text' section introduces reasoning tokens as our object of study and reviews the empirical evidence that the reasoning text is not a faithful explanation of the model's final answer. The 'The State over Tokens framework' section introduces the SoT framework formally. The 'Two misconceptions behind the illusion of explanation' section uses the framework to dispel two common misconceptions about the reasoning text. The 'The ontological divergence between text and state' section examines the divergence between the text and state interpretations of the same tokens. The 'Questions raised by state over tokens' section raises research questions opened by the framework.

Reasoning tokens and reasoning text

The subject of our discussion is the sequence of tokens that LLMs emit before their final answer, which we refer to as reasoning tokens (which we distinguish from reasoning text, which is the English interpretation of these same tokens). This approach started with prompting LLMs to generate reasoning tokens using examples or instructions such as 'let's think step by step'[11–13] but now models are trained to generate those tokens regardless of the input[3,4,14,15]; in some cases, step-level verifiers that score the steps by their semantic content are used to improve the generation[16–18]. Those tokens are often separated from the final answer given to the user. The amount of reasoning is variable and consequential: generating more tokens before answering tends to improve accuracy[19].

In this Perspective, we focus on characterizing the reasoning tokens. LLMs operate in the same way for the generation of the final answer as they do for the generation of reasoning tokens, but reasoning tokens differ from final-answer tokens in two specific ways that led us to focus our discussion on them.

The first reason is techno-sociological: reasoning tokens appear to offer a second product beyond the answer itself. When interpreted as English, the reasoning text reads like a description of reasoning work, encouraging readers to interpret them as not only what the model answers, but also how it seemingly arrived there, thereby providing a means of building trust[20,21]. This creates a conundrum specific to the reasoning tokens: the accuracy of the final answers keeps improving[2], while at the same time, the empirical evidence that the reasoning text fails to reflect how those answers were produced (reviewed in the 'Plausibility without faithfulness' section) continues to mount.

The second reason is functional: we can conceptually distinguish the two stages by their training objectives. The final-answer tokens are generated to be interpreted by humans (or external systems) as the solution to the user's query. During training, the answer text is optimized to reflect the solution, and thus has an additional constraint: its surface semantics must match human readers' semantics. By contrast, the reasoning tokens are not constrained in this way, and their purpose is to support producing the final answer. Indeed, their existence improves the accuracy of the final answer[3,15,19]. This distinction is especially pronounced in models trained with methods that do not subject the reasoning tokens to specific constraints as long as they serve the final prediction[4,22].

Plausibility without faithfulness

The reasoning text often reads like a logical derivation of the final answer. As Perrier[7] notes, it can correspond to an explanation according to some deductive proof system, representing a sequence of steps that leads to a conclusion. However, while this may count as an explanation of the answer, it is not an explanation of the model, and in particular it does not explain the process by which the model reached the answer.

Explanations are central to building trust in artificial intelligence models[20,21]. By explanation, we mean a description, at some useful level of abstraction, of a process by which the input can be transformed into the given output, identifying which factors were responsible for this output rather than another and allowing the user to anticipate how the output would change under counterfactual inputs[23,24].

https://doi.org/10.1038/s42256-026-01303-y

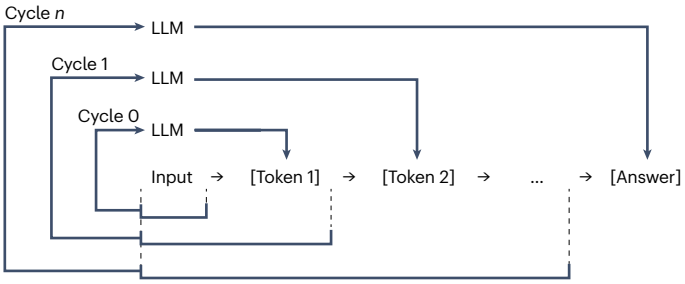


Fig. 1 | Illustration of token generation across cycles 0, 1, …, *n*, where the final answer is produced in the last cycle. In each cycle, the same LLM processes the input and previously generated tokens to produce the next token. The model maintains no internal memory between cycles that is not directly dependent on the tokens themselves, which carry the information forward. It helps to have language for this mechanism: tokens used as a computational device that enables the persistence of a process across separate and stateless computation cycles.

We take this causal, counterfactual-supporting notion of explanation as our working definition throughout the paper. An explanation can be plausible when it describes a process that could have led from the input to the output. An explanation is faithful when it reflects the process that led to the output, allowing the user to make accurate inferences about the model's process and reliable predictions of its behaviour on counterfactual or related inputs.

When treated as natural-language text, the reasoning text might seem like a faithful explanation, but this appearance is deceiving. The tokens invite a human reading: applying the rules of natural-language semantics to interpret them at face value. Through this act of reading, the reasoning text can yield a plausible explanation: one that describes a process that would lead from the input to the answer[7,25]. However, it often does not yield a faithful explanation[20]: the account describes merely a process consistent with its answer, not the actual process that led from the input to the output. This failure of faithfulness manifests as incompleteness of the text and a semantic mismatch between the LLM and the reader.

Incompleteness. The reasoning text can be incomplete and omit critical factors that affected the model's final answer[8,9,26–29]. A study conducted in controlled settings showed that LLMs can even seem to follow alignment goals while omitting the undesired topics from the reasoning text[30].

Semantic mismatch between LLM and reader. The semantic content of the reasoning text can be meaningless or misleading from a human perspective. Some studies have found that LLMs can disregard specific details in the text they generate[31–33]. Other work has shown that LLMs can be trained to produce irrelevant reasoning text and still provide accurate final answers[34–36]. This also leads to the failure to identify causal relations in the generation process of the reasoning text[37].

The peril lies in what we call the trust-calibration problem: the risk that plausibility can increase trust without justifying it, fostering over-reliance on model outputs. When reasoning text mimics systematic deliberation that includes certain factors that the users consider critical, this may lead them to place unwarranted trust in the model[38,39].

What can be said on reasoning tokens

Following this growing body of evidence and the practical risks it highlights, some have already argued that we should not treat reasoning text as a faithful explanation of the model's reasoning process[40–44]. However, we believe that we need to characterize the role of reasoning tokens in a positive way, beyond merely stating that they are not an explanation.

What can be said about the reasoning tokens? The following three statements hold.

- The reasoning tokens are an important part of the process that leads to the final answer.
- The reasoning tokens can sometimes be read as a series of steps that logically lead to the answer.
- The reasoning tokens are not a faithful explanation of the process that led to the final answer.

We would like a metaphor that accounts for all three of these points. In what follows, we provide the SoT view, which we believe does that.

The State over Tokens framework

Rather than explaining the reasoning tokens by how they are perceived by human readers (how they look), we consider their functional role within the mechanism of generation (what they do). We view reasoning tokens as an accumulated computational substrate—a medium through which the computation state is encoded in tokens.

The formal framework

Building on the intuition from the whiteboard analogy in the introduction—and consistent with how LLMs are implemented—we view the reasoning tokens not as a text with human-readable semantics, but rather as the evolving state of the reasoning process. The token sequence functions as a state for the model, although it can also be interpreted as natural-language text.

This state is what allows the LLM to maintain a coherent process across multiple computation cycles, each bounded in capacity, to solve problems that require more computation than any single cycle can provide.

To capture this architecture precisely, we view the autoregressive generation of the LLM as a recursive application of a pure function $\mathcal{M}(\cdot)$ on a token sequence (technically the LLM call is not pure as it involves a random sampling of tokens; we can assume that the random seed is part of the call to remedy this).

Each computation of $\mathcal{M}$ has limited computational capacity. While the computation capacity of the transformer[45] does increase slightly with each input token[46,47], this is a fixed increase that does not depend on the content of the tokens.

The function $\mathcal{M}(\cdot)$ is repeatedly applied to a sequence of inputs following a deterministic process. The initial input to the function is the user's input, a sequence of tokens we denote as S_0 (commonly referred to as the context in LLM literature; we use input to emphasize its functional role as the argument to $\mathcal{M}$):

$$S_0 = \text{user input}$$

At each subsequent cycle k, the function $\mathcal{M}$ takes the current sequence S_k as its input and produces a new token as output. This token is appended to S_k to form S_{k+1}, the input for the next call:

$$S_{k+1} = S_k \oplus \mathcal{M}(S_k)$$

Here, $\oplus$ denotes the concatenation operation. This continues until the final answer is produced. Note that each input (a computation state) of one of the previous cycles is a prefix of S_k. Figure 1 illustrates this recursive-generation process across cycles.

The function of the tokens. This formalization reveals three key observations about the functional role of the tokens.

- The tokens are what allows persistence across cycles. The tokens are the carriers of state, and hence the sole facilitators of persistency, across cycles. As the model itself remains unchanged across cycles, it is the concatenation of the output of each cycle to the input of the next that allows the model to reach more

advanced results in subsequent computations, until finally arriving at the final output. In the decoder architecture used by current LLMs[48,49], some internal activations also persist between cycles, but the tokens are the only non-redundant information that persists. When generating token $k+1$, the representations at the position of the newly added token k must be computed from scratch (as it was the output of the previous cycle and has no prior internal representation). The representations at positions $1 \ldots k-1$ remain the same and are often retained via the 'KV cache' optimization to avoid redundant computation, but they encode no information beyond what the token prefix already determines.

- The tokens dictate exclusively future computation. The computation of $\mathcal{M}$ is fully determined by the sequence S_k.
- Encoding and decoding are internal to the LLM. The way that S_k affects the next computation and the overall process that leads to the answer is dependent on $\mathcal{M}$.

The process the tokens enable. The tokens are what enable the mechanism of a process that is composed of multiple cycles. Each individual computation is limited—it can perform only as much work as the LLM's depth allows in a single cycle. However, by encoding results into S_k and building on them in subsequent cycles, the computational capacity of multiple computations of the LLM is utilized. The accumulation of tokens transforms the process from one bounded by a single cycle to one that utilizes computation performed across multiple cycles. This conceptual view is supported by theoretical findings, which demonstrate that—at least for specific constructions—accumulating tokens formally increases computational capacity[50,51].

Following those observations, we argue that an accurate and revealing name for this is State over Tokens (SoT). We use this term for the remainder of the paper.

Implications of the SoT view

Viewing the reasoning tokens as a state means that we can attribute to them the properties inherent to the concept of 'state' in computation. This reframing has several immediate implications for how we understand the tokens.

A state is forward-looking. A state enables future computation, not describing past computations. While it is created by past computation, it does not determine it: various computations can lead to the same state. Unlike a log that records past cycles, a state encodes what is needed to continue the process.

A change in the state will lead to a different process. Altering the state will change the trajectory of future computations.

States are discrete snapshots. The tokens seem to describe a flowing narrative, a text. But the LLM does not process this flow—at each cycle, it operates on a single prefix as a discrete state. Reading continuously obscures how each prefix functions in the process, which may differ from what the tokens appear to mean as a narrative.

A state is necessarily partial. As a state, the tokens need only contain what is required for the next computation to proceed with the overall process. The bulk of each cycle's computation does not need to be externalized (see 'The misconception of completeness' section for further discussion).

A state is created to be used by its creator. What matters is not what a generated token means to human readers, but how the LLM decodes and uses it in subsequent cycles. The LLM can use its own semantics independent of natural language interpretation (see 'The misconception of shared meaning' section).

Two misconceptions behind the illusion of explanation

To examine the nature of the relation between a sequence of states and the computation that produced it, it is instructive to take a step back and consider recursive iterative processes where $\mathcal{M}$ is not an LLM but a simpler function.

As an example, consider a process that computes the Fibonacci numbers, a sequence of natural numbers occurring in various counting problems. The Nth Fibonacci number is defined recursively by the formula $F_n = F_{n-1} + F_{n-2}$ with base cases $F_1 = F_2 = 1$. The recursive definition can be translated into an iterative one, in which a pure function takes a prefix of the sequence and computes the next value according to the recursive definition. When each number is computed, it is added to the state that becomes the input of the next function call. The function $\mathcal{M}$ takes as input a target index N and a prefix of k values (the first k Fibonacci numbers), and returns the $(k+1)$ Fibonacci number. The computation state in our case is the sequence of k already computed numbers. Each computed value is concatenated to the input sequence; the resulting sequence becomes the state for the next call. For example, computing the 6th Fibonacci number generates the sequence of states:

$$
\begin{aligned}
S_0 &= 6\text{th?} \\
S_1 &= S_0 \oplus \mathcal{M}(S_0) = S_0 \oplus \mathcal{M}(6\text{th?}) = 6\text{th?} \oplus 1 \\
S_2 &= S_1 \oplus \mathcal{M}(S_1) = S_1 \oplus \mathcal{M}(6\text{th?}, 1) = 6\text{th?}\,1 \oplus 1 \\
S_3 &= S_2 \oplus \mathcal{M}(S_2) = S_2 \oplus \mathcal{M}(6\text{th?}, 1, 1) = 6\text{th?}\,1, 1 \oplus 2 \\
S_4 &= S_3 \oplus \mathcal{M}(S_3) = S_3 \oplus \mathcal{M}(6\text{th?}, 1, 1, 2) = 6\text{th?}\,1, 1, 2 \oplus 3 \\
S_5 &= S_4 \oplus \mathcal{M}(S_4) = S_4 \oplus \mathcal{M}(6\text{th?}, 1, 1, 2, 3) = 6\text{th?}\,1, 1, 2, 3 \oplus 5 \\
S_6 &= S_5 \oplus \mathcal{M}(S_5) = S_5 \oplus \mathcal{M}(6\text{th?}, 1, 1, 2, 3, 5) = 6\text{th?}\,1, 1, 2, 3, 5 \oplus 8
\end{aligned}
$$

Note that while the numbers 1, 1, 2, 3 and 5 are all needed for computing 8, and were used in the process of computing 8, we intuitively do not consider them as an explanation for the answer 8 without the additional information that an addition operation was used on them. But why would it be different for the intermediate steps of an LLM? This form of alienation is already instructive. We consider below two concrete implications.

The misconception of completeness

It should be clear to any reader that while the intermediate sequence 1, 1, 2, 3, 5 is crucial for computing 8, this sequence results from the computation and allows it to continue, but it is not the computation itself. The sequence of values does not equal the computation that took place for creating them. In addition, this sequence does not determine a computation: we cannot infer the computation from the sequence (we can guess but cannot be sure), and it is clear that the computation steps that were involved in computing each number are not reflected in the output sequence. We see results of intermediate computations, not the algorithm that is used for producing them. Consider a practical example: from the text 1, 1, 2, 3, 5, 8 alone, and without prior knowledge of the Fibonacci recurrence, we could not reliably predict what the model would output if asked for the 7th item in the sequence, whereas a complete explanation would also communicate the computation performed between each pair of tokens, at some useful level of abstraction, so that we would be able to compute the answer $\mathcal{M}$ would provide if asked for the 7th item in the sequence.

We can also imagine a function $\mathcal{M}'$ that is called only 3 times and not 6, with the intermediate values 1, 3, 8. This function computes two following numbers at a time, and outputs only the second one. In each invocation, it recomputes the missing numbers, and continues the sequence. Even if we do choose to treat the steps as an explanation, there is no guarantee that this explanation is complete.

The SoT functions as scaffolding that propels the process forward; a misconception arises when we mistake this scaffolding for the building.

The misconception of shared meaning

Even if we accept that the state shows only partial results, we might assume that $\mathcal{M}$ interprets them as humans do. However, this assumption is also mistaken.

Consider an alternative function for computing the Nth Fibonacci number, where the state that was used to produce the answer 8 is not 1, 1, 2, 3, 5, 8 but rather 11, 11, 12, 13, 15, 18. Such a function could be just as effective: at each step, $\mathcal{M}$ will subtract 10 from each of its inputs, perform the computation, add 10 to the result, and return it as output. At the final (Nth) stage, it will subtract 10 from the last item and report it as the answer.

This simple additive transformation demonstrates that the reasoning tokens in the SoT—the very symbols we read and interpret—may function in a way entirely different from what human readers naturally ascribe to them. Even elementary encoding schemes can render the surface meaning of text opaque to human interpretation while remaining fully usable for $\mathcal{M}$. Consider a practical example: if we ask a reader what the model would output for a query about the 5th item in the sequence, the reader, interpreting these symbols at face value, would predict 15—the 5th visible value—while $\mathcal{M}$, under its additive-shift encoding, would output 5.

Beyond numerical encoding. The two misconceptions surfaced above through the simple $\mathcal{M}$ apply directly to LLMs, where they can be even more pronounced: LLM processes are far more complex than the toy $\mathcal{M}$, and their outputs are not restricted to numerical encoding. When an LLM writes 'I need to reconsider this approach', the phrase might function not as genuine metacognitive reflection, but as an encoding for something else, for example, a mathematical computation. Given empirical findings on how models can embed functionally relevant information in ways that remain opaque to human readers[52], it is likely that real-world encodings are far more intricate than a simple additive shift. This view, together with 'The misconception of completeness', may also give intuition to the phenomenon where LLMs succeed at evading mentioning relevant topics in their reasoning text[53,54].

The ontological divergence between text and state

We argue that the trust-calibration problem arises from an ontological divergence between form and function in reasoning LLMs, a distinctive case of dual interpretation. By divergence we mean that the same tokens can be interpreted (by humans) as English text while at the same time being interpreted and used (by the LLM) as an encoding of a computation state for an iterative function. Symbols admitting multiple interpretations are common: the word 'apple' can refer to either a fruit or a company; the same sequence of bytes can encode either a floating-point number or a memory address. But in this case, two additional features set the reasoning tokens apart. The first is that through one interpretation, the symbols look like they provide information on a computational process that uses the symbols by their other interpretation (by the LLM). In other words, the process that decides what symbol will be next is the same process that the symbols appear to describe. The second is that the two interpretations of the symbols are not constrained to match, as discussed in 'Two misconceptions behind the illusion of explanation'.

The joint presence of these two features is what creates the trust-calibration problem. To a human reader, the sequence is a text, parsed according to linguistic conventions to extract semantic meaning. To the model, however, the same tokens serve as a computational substrate, accumulated one token at a time to mechanically drive the next step of a process. These two roles, a communicative medium and a functional state, happen to share the same surface form, but neither one is constrained by the other. The LLM uses a sequence of tokens that can be interpreted by humans according to one semantic system, while at the same time it is interpreted by the LLM using an alternative semantic system. This is how the tokens can function as a computational substrate without becoming about the computation in a way that human readers can discern by reading. We do not rule out that reasoning tokens may function as faithful explanations in some models, tasks or training regimes. The point is that their natural-language appearance is not sufficient evidence of faithfulness; treating them as faithful explanations requires positive evidence that they support accurate causal and counterfactual inferences about the model's computation.

Questions raised by state over tokens

What SoT means for interpretability

Rather than solving the challenges that existing interpretability work addresses—such as understanding model internals, tracing how features emerge or identifying mechanisms[55–57]—the SoT view surfaces a distinct, new challenge: investigating how LLMs construct and use computational state through tokens. We call for recognizing the encoded state as a first-class object of study. This approach neither reads the reasoning tokens as ordinary English text nor dismisses them as an inadequate explanation, but rather acknowledges that understanding how LLMs use state requires work to decode the structure and function of the token sequence. The new overarching question becomes 'how is the computation state encoded in the tokens being maintained and used by the LLM?' This shift in perspective opens new questions about the structure and behaviour of the encoded state itself.

- How do LLMs decide what information to externalize at each cycle?
- What information is actually encoded in the state at different points in the sequence?
- Do they use consistent encodings across the solution of a problem and across different problems?
- How does information propagate through the sequence?

Understanding these properties requires decoding the relationship between internal computation and the externalized token-based state.

Recent interpretability work provides a starting point—for instance, studying which parts of the sequence are most critical for the final answer[58] or analysing model components' activity during different cycles[59,60]. However, the grand challenge of understanding how LLMs manage state through tokens remains largely open.

The role of language in SoT

The phenomenon of reasoning tokens is based on models that already had the ability to generate fluent natural language; in this sense, it can be seen as an exploitation of an existing capability rather than a designed-in feature. Chain-of-thought prompting[12], scratchpads[11], the zero-shot 'let's think step by step' instruction[13] and STaR[3] are widely regarded as the works that established the technique of making the model produce tokens before the final answer. These works treat the medium from a human perspective, eliciting reasoning tokens through input semantics[12,13] or by training the model on rationales[3,4,11]. A subsequent line of work pushes this further by directly leveraging the natural-language form of intermediate steps, through step-level process reward models[16] and majority voting over sampled sequences of tokens[61]. Together, these works show how natural language, the central capability of LLMs before they were trained to generate reasoning tokens, has been turned into a substrate for iterative computation. This suggests that carrying the state in natural-language tokens is inherited from LLMs' origins as language models rather than required by state-based computation itself.

Looking at this phenomenon through SoT raises the question of the arbitrariness of the medium for discrete, token-based computation.

We previously argued that it can, in principle, encode arbitrary computation regardless of its surface semantics. But to what extent is this actually the case? Are all media equally expressive, or is using natural language sequences particularly well suited for encoding the kinds of computations that LLMs perform?

One can hypothesize that the massive pretraining stage on natural language data induces an inductive bias to reason[62] in ways that are in accordance with its semantics, which makes the reasoning text read as a plausible explanation. Under this view, elaborate encoding schemes may either contradict the training distribution—where SoT updates often gradually lead to the correct continuation—or require additional computation that is disfavoured given the pretraining data. If this hypothesis is correct, LLMs possess at least an inductive bias to represent state in a manner that reflects their internal computation.

In a different direction, recent studies have explored alternative media for state representation with latent reasoning ranging from simple vectors[63–66] to structured formats[67]. However, the question of whether natural language offers unique advantages over these alternatives remains open. We note that, by representing state in a medium that is not readable as natural language, this family of approaches sidesteps the trust-calibration problem identified above—the state cannot be misread as an explanation, as no surface natural-language reading is invited.

The prospects for faithful reasoning text

Finally, the SoT view sharpens the question of whether the faithfulness of reasoning tokens can be improved. If we aim to make the text a faithful explanation, a tension may arise: the reasoning tokens will have to satisfy two distinct objectives. Functionally, as a state, the tokens are optimized to encode the information necessary to drive the next computation cycles. Interpretively, as an explanation, we want them to also describe the computation itself—whether they describe the computation that has already occurred in past cycles, or the computation that will occur in future cycles.

It is not immediately obvious that this dual requirement can be satisfied without loss of performance. If the tokens must serve as both state and explanation then they must simultaneously encode the state and describe the computation. Requiring an LLM to explain its own computation while simultaneously performing it amounts to a form of metacognitive capability; the LLM must reason about its own reasoning process within the same sequence that sustains it. For the explanation to be faithful, the information the model encodes to solve the problem (the state) must be communicated in a way that a human can extract by reading the text (the explanation). However, an optimal computational state might require encoding information that is redundant, nonlinear or semantically opaque to a human reader. Conversely, constraining the state to be a coherent, linear English narrative might strip it of the information needed for the reasoning process. In relation to this, Guan et al.[68] found that, in the settings they tested and under a fixed token budget, increasing the faithfulness of the reasoning text comes at the cost of final-answer accuracy. In a similar direction, Bhambri et al.[69] found that fine-tuning on the reasoning traces that human readers judged least interpretable yielded the strongest performance. Conversely, in the related setting of concept-based models, coupling the explanation to the representations the model actually uses makes explanation-based interventions more effective[70], suggesting that alignment between the two objectives could also serve as a useful training signal. Together, these findings leave open the question of whether the two objectives are reconcilable.

Thus, the challenge is not merely whether LLMs can be trained to produce plausible explanations, but whether the medium of natural language tokens has the capacity to simultaneously function as an efficient computational substrate and a transparent descriptive record. This forces us to confront a fundamental question: can the same sequence of symbols simultaneously carry the full weight of a machine's computation and transparently reveal the logic of that process to humans?

The question extends also to viewing reasoning texts as rationalizations: texts that consist of a sequence of steps leading to the answer, which can be used by a human reader to verify it. To what extent can a sequence be used concurrently both as carrying state and as providing an effective rationalization?

Conclusion

We have argued that the SoT view characterizes the reasoning tokens generated by LLMs before an answer not by their appearance but by their role as an externalized computational state.

Adopting this perspective resolves the persistent misconceptions that underlie the illusion of explanation. The misconception of completeness is explained by recognizing that SoT can externalize only what is functionally necessary for the next cycle, not a full account of the computation that occurred. The misconception of shared meaning is exposed by the insight that the text's human-readable meaning need not coincide with its computational function, which may rely on arbitrary, model-specific encodings. This semantic mismatch—where the token sequence functions as a computational substrate while appearing as natural language—is a distinctive case of ontological divergence. A fluent natural-language reading appears to describe the very computation it drives, with no constraint that the two coincide, which creates the trust-calibration problem: a plausible-looking explanation can be mistaken for a faithful one.

While much research has focused on what this text is not, the SoT view attempts to explain what it is. This opens concrete research directions—understanding how LLMs construct and use state through tokens, investigating whether natural language possesses unique advantages as a computational medium, and exploring whether SoT can be made to faithfully explain the underlying computation. Recognizing the reasoning tokens as a computational state provides a clear basis for calibrating trust and for investigating the questions this view opens.

References

1. Mitchell, M. Artificial intelligence learns to reason. *Science* **387**, eadw5211 (2025).
2. Maslej, N. et al. Artificial intelligence index report 2025. Preprint at https://doi.org/10.48550/arXiv.2504.07139 (2025).
3. Zelikman, E., Wu, Y., Mu, J. & Goodman, N. D. STaR: bootstrapping reasoning with reasoning. *Adv. Neural Inf. Process. Syst.* **35**, 15476–15488 (2022).
4. DeepSeek-AI, Guo, D., Yang, D. & Zhang, H. et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* **645**, 633–638 (2025).
5. Zheng, C. et al. ProcessBench: identifying process errors in mathematical reasoning. In *Proc. 63rd Annual Meeting of the Association for Computational Linguistics* (eds Che, W. et al.) https://aclanthology.org/2025.acl-long.50/ (ACL, 2025).
6. Lee, J. & Hockenmaier, J. Evaluating step-by-step reasoning traces: a survey. In *Findings of the Association for Computational Linguistics* 1789–1814 https://aclanthology.org/2025.findings-emnlp.94/ (ACL, 2025).
7. Perrier, E. Typed chain-of-thought: a Curry–Howard framework for verifying LLM reasoning. Preprint at https://doi.org/10.48550/arXiv.2510.01069 (2025).
8. Chua, J. & Evans, O. Are DeepSeek R1 and other reasoning models more faithful? Preprint at https://doi.org/10.48550/arXiv.2501.08156 (2025).
9. Cornish, C. & Rogers, A. Examining the faithfulness of DeepSeek R1's chain-of-thought reasoning. In *Proc. 1st Workshop on Confabulation, Hallucinations and Overgeneration in Multilingual and Practical Settings* (eds Sinha, A. et al.) 11–19 (ACL, 2025).

10. Lakoff, G. & Johnson, M. *Metaphors We Live By* (Univ. Chicago Press, 1980).
11. Nye, M. et al. Show your work: scratchpads for intermediate computation with language models. Preprint at https://doi.org/10.48550/arXiv.2112.00114 (2021).
12. Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **35**, 24824–24837 (2022).
13. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. & Iwasawa, Y. Large language models are zero-shot reasoners. *Adv. Neural Inf. Process. Syst.* **35**, 22199–22213 (2022).
14. OpenAI o1 System Card. *OpenAI* https://openai.com/index/openai-o1-system-card/ (2024).
15. Comanici, G. et al. Gemini 2.5: pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. Preprint at https://doi.org/10.48550/arXiv.2507.06261 (2025).
16. Wang, P. et al. Math-Shepherd: verify and reinforce llms step-by-step without human annotations. In *Proc. 62nd Annual Meeting of the Association for Computational Linguistics* https://aclanthology.org/2024.acl-long.510/ (ACL, 2024).
17. Guan, X. et al. rStar-Math: small LLMs can master math reasoning with self-evolved deep thinking. In *Proc. 42nd International Conference on Machine Learning* (eds Singh, A. et al.) https://openreview.net/forum?id=5zwF1GizFa (PMLR, 2025).
18. Zhang, Z. et al. The lessons of developing process reward models in mathematical reasoning. In *Findings of the Association for Computational Linguistics* (eds Che, W. et al.) https://aclanthology.org/2025.findings-acl.547/ (ACL, 2025).
19. Muennighoff, N. et al. s1: simple test-time scaling. In *Proc. 2025 Conference on Empirical Methods in Natural Language Processing* 20275–20321 https://aclanthology.org/2025.emnlp-main.1025/ (ACL, 2025).
20. Jacovi, A., Marasović, A., Miller, T. & Goldberg, Y. Formalizing trust in artificial intelligence: prerequisites, causes and goals of human trust in AI. In *Proc. 2021 ACM Conference on Fairness, Accountability, and Transparency* (eds Elish, M. C. et al.) 624–635 (ACM, 2021).
21. Ferrario, A. & Loi, M. How explainability contributes to trust in AI. In *Proc. 2022 ACM Conference on Fairness, Accountability, and Transparency* https://api.semanticscholar.org/CorpusID:246458160 (ACM, 2022).
22. Lambert, N. et al. Tulu 3: Pushing frontiers in open language model post-training. Preprint at https://doi.org/10.48550/arXiv.2411.15124 (2024).
23. Woodward, J. *Making Things Happen: A Theory of Causal Explanation* (Oxford Univ. Press, 2003).
24. Miller, T. Explanation in artificial intelligence: insights from the social sciences. *Artif. Intell.* **267**, 1–38 (2019).
25. Samineni, S. R., Kalwar, D., Gangal, V., Bhambri, S. & Kambhampati, S. Local coherence or global validity? Investigating RLVR traces in math domains. Preprint at https://doi.org/10.48550/arXiv.2510.18176 (2025).
26. Lindsey, J. et al. On the biology of a large language model. *Transformer Circuits Thread* https://transformer-circuits.pub/2025/attribution-graphs/biology.html (2025).
27. Arcuschin, I. et al. Chain-of-thought reasoning in the wild is not always faithful. Preprint at https://doi.org/10.48550/arXiv.2503.08679 (2025).
28. Marioriyad, A., Adim, S., Alighardashi, N., Banghshah, M. S. & Rohban, M. H. Unspoken hints: accuracy without acknowledgement in LLM reasoning. Preprint at https://doi.org/10.48550/arXiv.2509.26041 (2025).
29. Young, R. J. Lie to me: how faithful is chain-of-thought reasoning in reasoning models? Preprint at https://doi.org/10.48550/arXiv.2603.22582 (2026).
30. Skaf, J. et al. Large language models can learn and generalize steganographic chain-of-thought under process supervision. Preprint at https://doi.org/10.48550/arXiv.2506.01926 (2025).
31. Chen, Y. et al. Reasoning models don't always say what they think. Preprint at https://doi.org/10.48550/arXiv.2505.05410 (2025).
32. Nikankin, Y., Tutek, M., Ashuach, T., Rosenfeld, J. & Belinkov, Y. Reasoning models know what's important, and encode it in their activations. Preprint at https://doi.org/10.48550/arXiv.2604.18307 (2026).
33. Boppana, S. et al. Reasoning theater: disentangling model beliefs from chain-of-thought. Preprint at https://doi.org/10.48550/arXiv.2603.05488 (2026).
34. Stechly, K., Valmeekam, K., Gundawar, A., Palod, V. & Kambhampati, S. Beyond semantics: the unreasonable effectiveness of reasonless intermediate tokens. Preprint at https://doi.org/10.48550/arXiv.2505.13775 (2025).
35. Bhambri, S., Biswas, U. & Kambhampati, S. Interpretable traces, unexpected outcomes: investigating the disconnect in trace-based knowledge distillation. Preprint at https://doi.org/10.48550/arXiv.2505.13792 (2025).
36. Zolkowski, A., Nishimura-Gasparian, K., McCarthy, R., Zimmermann, R. S. & Lindner, D. Early signs of steganographic capabilities in frontier LLMs. Preprint at https://doi.org/10.48550/arXiv.2507.02737 (2025).
37. Levy, M., Elyoseph, Z. & Goldberg, Y. Humans perceive wrong narratives from AI reasoning texts. Preprint at https://doi.org/10.48550/arXiv.2508.16599 (2025).
38. Ehsan, U. & Riedl, M. O. Explainability pitfalls: beyond dark patterns in explainable AI. *Patterns* **5**, 100971 (2024).
39. Rosenbacke, R., Melhus, Å, McKee, M. & Stuckler, D. How explainable artificial intelligence can increase or decrease clinicians' trust in AI applications in health care: systematic review. *JMIR AI* **3**, e53207 (2024).
40. Agarwal, C., Tanneru, S. H. & Lakkaraju, H. Faithfulness vs. plausibility: on the (un) reliability of explanations from large language models. Preprint at https://doi.org/10.48550/arXiv.2402.04614 (2024).
41. Sarkar, A. Large language models cannot explain themselves. Preprint at https://doi.org/10.48550/arXiv.2405.04382 (2024).
42. Barez, F. et al. Chain-of-thought is not explainability. Preprint at https://www.alphaxiv.org/abs/2507.chain-of-thought-is-not-explainability (2025).
43. Kambhampati, S. et al. Position: stop anthropomorphizing intermediate tokens as reasoning/thinking traces! Preprint at https://doi.org/10.48550/arXiv.2504.09762 (2025).
44. Hu, Y. "Thinking traces" in large reasoning models: cognitive cost or performative scaffolding? *Proc. Natl Acad. Sci. USA* **123**, e2604554123 https://doi.org/10.1073/pnas.2604554123 (2026).
45. Vaswani, A. et al. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, 5998–6008 (2017).
46. Merrill, W., Sabharwal, A. & Smith, N. A. Saturated transformers are constant-depth threshold circuits. *Trans. Assoc. Comput. Linguist.* **10**, 843–856 (2022).
47. Pfau, J., Merrill, W. & Bowman, S. R. Let's think dot by dot: hidden computation in transformer language models. Preprint at https://doi.org/10.48550/arXiv.2404.15758 (2024).
48. Liu, P. J. et al. Generating Wikipedia by summarizing long sequences. In *International Conference on Learning Representations* https://openreview.net/forum?id=Hyg0vbWC- (ICLR, 2018).
49. Radford, A. & Narasimhan, K. Improving language understanding by generative pre-training. *Semantic Scholar* https://api.semanticscholar.org/CorpusID:49313245 (2018).
50. Merrill, W. & Sabharwal, A. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations* (eds Kim, B., et al.) 7690–7706 (ICLR, 2024).

51. Li, Z., Liu, H., Zhou, D. & Ma, T. Chain of thought empowers transformers to solve inherently serial problems. In *12th International Conference on Learning Representations* (eds Kim, B. et al.) https://openreview.net/forum?id=3EWTEy9MTM (ICLR, 2024).
52. Cloud, A. et al. Subliminal learning: language models transmit behavioral traits via hidden signals in data. Preprint at https://doi.org/10.48550/arXiv.2507.14805 (2025).
53. Emmons, S. et al. When chain of thought is necessary, language models struggle to evade monitors. Preprint at https://doi.org/10.48550/arXiv.2507.05246 (2025).
54. Li, C., Phuong, M. & Siegel, N. Y. LLMs can covertly sandbag on capability evaluations against chain-of-thought monitoring. Preprint at https://doi.org/10.48550/arXiv.2508.00943 (2025).
55. Alain, G. & Bengio, Y. Understanding intermediate layers using linear classifier probes. *Stat* **1050**, 14 (2016).
56. Li, K. et al. Emergent world representations: exploring a sequence model trained on a synthetic task. In *International Conference on Learning Representations* 1–17 (ICLR, 2023).
57. Bereska, L. & Gavves, E. Mechanistic interpretability for AI safety—a review. *Transactions on Machine Learning Research* https://openreview.net/forum?id=ePUVetPKu6 (2024).
58. Bogdan, P. C., Macar, U., Nanda, N. & Conmy, A. Thought anchors: which LLM reasoning steps matter? Preprint at https://doi.org/10.48550/arXiv.2506.19143 (2025).
59. Chen, X., Plaat, A. & van Stein, N. How does chain of thought think? Mechanistic interpretability of chain-of-thought reasoning with sparse autoencoding. Preprint at https://doi.org/10.48550/arXiv.2507.22928 (2025).
60. Zhou, Y., Wang, Y., Yin, X., Zhou, S. & Zhang, A. R. The geometry of reasoning: flowing logics in representation space. Preprint at https://doi.org/10.48550/arXiv.2510.09782 (2025).
61. Wang, X. et al. Self-consistency improves chain of thought reasoning in language models. In *11th International Conference on Learning Representations* https://openreview.net/forum?id=1PL1NIMMrw (ICLR, 2023).
62. Venhoff, C., Arcuschin, I., Torr, P., Conmy, A. & Nanda, N. Base models know how to reason, thinking models learn when. Preprint at https://doi.org/10.48550/arXiv.2510.07364 (2025).
63. Hao, S. et al. Training large language models to reason in a continuous latent space. In *2nd Conference on Language Modeling* https://openreview.net/forum?id=Itxz7S4Ip3 (OpenReview, 2025).
64. Butt, N., Kwiatkowski, A., Labiad, I., Kempe, J. & Ollivier, Y. Soft tokens, hard truths. In *International Conference on Learning Representations* (eds Vondrick, C. et al.) https://openreview.net/forum?id=9JjKTp8Jmy (ICLR, 2026).
65. Hwang, H. et al. Latent reasoning via sentence embedding prediction. Preprint at https://doi.org/10.48550/arXiv.2505.22202 (2025).
66. Zhu, R.-J. et al. A survey on latent reasoning. Preprint at https://doi.org/10.48550/arXiv.2507.06203 (2025).
67. Domingos, P. Tensor logic: the language of AI. Preprint at https://doi.org/10.48550/arXiv.2510.12269 (2025).
68. Guan, M. Y. et al. Monitoring monitorability. Preprint at https://doi.org/10.48550/arXiv.2512.18311 (2025).
69. Bhambri, S., Biswas, U. & Kambhampati, S. Do cognitively interpretable reasoning traces improve LLM performance? Preprint at https://doi.org/10.48550/arXiv.2508.16695 (2025).
70. Zabounidis, R. et al. Disentangled concept-residual models: bridging the interpretability—performance gap for incomplete concept sets. *Transactions on Machine Learning Research* https://openreview.net/forum?id=NKgNizwDa6 (2026).

Author contributions

M.L. wrote the original draft. M.L., Z.E., S.R. and Y.G. developed the conceptual framework, reviewed and edited the paper, and approved the final version.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence should be addressed to Mosh Levy.

Peer review information *Nature Machine Intelligence* thanks Wenxiang Jiao, Jingcheng Niu and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© Springer Nature Limited 2026